Звіт до лабораторної роботи №2

Тема: Класи, конструктори, властивості з валідацією, деструктор і збирання сміття (C#) **Варіант:** 7 **Виконав:** Зінчук Д. Ю.

1. Посилання на репозиторій

GitHub: https://github.com/zinchukdiipz25-wq/oop-ipz3-1-zinchuk/blob/main/Program.cs

2. Постановка завдання

Реалізувати клас `Phone` з полями, властивостями з валідацією, двома перевантаженими конструкторами з ланцюговим викликом, методом і деструктором (фіналізатором). Продемонструвати створення кількох об'єктів різними конструкторами, виклик методів і роботу збирача сміття (GC).

3. Код програми

```
using System;

class Phone
{
    // Приватні поля
    private string _brand;
    private string _model;
    private int _batteryLevel;

    // Властивість Brand
    public string Brand
    {
        get { return _brand; }
        set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Бренд не може бути порожнім!");

            _brand = value;
        }
    }

    // Властивість Model
    public string Model
```

```
    {
        get { return _model; }
        set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Модель не може бути
порожньою!");

            _model = value;
        }
    }

    // Властивість BatteryLevel з валідацією
    public int BatteryLevel
    {
        get { return _batteryLevel; }
        set
        {
            if (value < 0 || value > 100)
                throw new ArgumentOutOfRangeException(
                    nameof(value), "Заряд має бути від 0 до 100!");

            _batteryLevel = value;
        }
    }

    // Конструктор без параметрів
    public Phone() : this("Generic", "Basic", 50)
    {
        Console.WriteLine("Створено телефон за замовчуванням.");
    }

    // Параметризований конструктор
    public Phone(string brand, string model, int batteryLevel)
    {
        Brand = brand;
        Model = model;
        BatteryLevel = batteryLevel;

        Console.WriteLine($"Створено телефон: {Brand} {Model}");
    }

    // Метод дзвінка
    public void Call(string number)
    {
        if (string.IsNullOrWhiteSpace(number))
```

```
        {
            Console.WriteLine("Помилка: номер не вказано!");
            return;
        }

        if (BatteryLevel == 0)
        {
            Console.WriteLine($"{Brand}: недостатньо заряду!");
            return;
        }

        Console.WriteLine($"{Brand} {Model} телефонує на {number}.");

        BatteryLevel = Math.Max(0, BatteryLevel - 10);

        Console.WriteLine($"Залишок заряду: {BatteryLevel}%");
    }

    // Фіналізатор
    ~Phone()
    {
        Console.WriteLine($"Об'єкт {Brand} {Model} знищено.");
    }
}

class Program
{
    static void Main()
    {
        Console.WriteLine("=== Телефон 1 ===");
        Phone phone1 = new Phone();

        Console.WriteLine();

        Console.WriteLine("=== Телефон 2 ===");
        Phone phone2 = new Phone("Apple", "iPhone 12", 80);

        Console.WriteLine();

        Console.WriteLine("=== Телефон 3 ===");
        Phone phone3 = new Phone("Samsung", "Galaxy S24", 20);

        Console.WriteLine();

        phone1.Call("0671234567");
        Console.WriteLine();
```

```
        phone2.Call("0509876543");
        Console.WriteLine();

        phone3.Call("0631112233");
        Console.WriteLine();

        Console.WriteLine("=== Примусове збирання сміття ===");

        // Навчальна демонстрація фіналізатора
        phone1 = null;
        phone2 = null;
        phone3 = null;

        GC.Collect();
        GC.WaitForPendingFinalizers();

        Console.WriteLine("Програму завершено.");
    }
}
```

4. Опис класу

Клас `Phone` містить три приватні поля: `_brand`, `_model` та `_batteryLevel`. Доступ до них здійснюється через властивості з валідацією. `Brand` і `Model` не приймають порожніх рядків та рядків із самих пробілів, інакше генерується `ArgumentException`. `BatteryLevel` приймає лише значення від 0 до 100, інакше генерується `ArgumentOutOfRangeException`.

Конструктори:

- `Phone()` без параметрів через ланцюговий виклик `this("Generic", "Basic", 50)` передає керування параметризованому конструктору зі значеннями за замовчуванням і лише після цього виводить власне повідомлення;
- `Phone(string brand, string model, int batteryLevel)` присвоює значення через властивості, тому валідація спрацьовує вже під час створення об'єкта.

Метод `Call(string number)` перевіряє номер і заряд, виводить інформацію про дзвінок та зменшує заряд на 10% (не нижче 0). Фіналізатор `~Phone()` виводить повідомлення в момент знищення об'єкта збирачем сміття.

5. Результат виконання програми

Нижче наведено очікуваний вивід консолі. Порядок рядків «знищено» не гарантований, бо його визначає збирач сміття.

```
=== Телефон 1 ===
Створено телефон: Generic Basic
Створено телефон за замовчуванням.

=== Телефон 2 ===
Створено телефон: Apple iPhone 12

=== Телефон 3 ===
Створено телефон: Samsung Galaxy S24

Generic Basic телефонує на 0671234567.
Залишок заряду: 40%

Apple iPhone 12 телефонує на 0509876543.
Залишок заряду: 70%

Samsung Galaxy S24 телефонує на 0631112233.
Залишок заряду: 10%

=== Примусове збирання сміття ===
Об'єкт Samsung Galaxy S24 знищено.
Об'єкт Apple iPhone 12 знищено.
Об'єкт Generic Basic знищено.
Програму завершено.
```

Місце для скриншота консолі: вставте власний скриншот після запуску програми.

6. Висновок

У ході роботи було реалізовано клас `Phone` з інкапсульованими полями, властивостями з валідацією, двома конструкторами з ланцюговим викликом, методом `Call` та фіналізатором. Демонстрація показала такий життєвий цикл об′єкта: спочатку виділяється пам′ять і виконується ланцюг конструкторів (спершу параметризований, потім тіло конструктора без параметрів), далі об′єкт використовується через свої методи, а після втрати всіх посилань (`phone = null`) він стає кандидатом на видалення.

У C# звільненням пам'яті керує збирач сміття, тому момент виклику фіналізатора невизначений і не залежить від програміста. У програмі його викликано примусово через `GC.Collect()` та `GC.WaitForPendingFinalizers()`, і це доречно лише для навчальної демонстрації. На практиці примусовий виклик GC не рекомендується, а для вивільнення некерованих ресурсів краще застосовувати інтерфейс `IDisposable` та оператор `using`.